Collaborative Filtering & Recommender Systems

A notation-first study guide, with the cost function taken apart term by term — built from Andrew Ng's Machine Learning Specialization



Coral Teal Amber Blue Purple Orange Green

Inside: the full notation table, every cost function built up piece by piece with a hand-worked numeric example, the running Alice/Bob/Carol/Dave movie-rating example throughout, gradient descent updates, the binary-labels extension, and interview-style Q&A.

Sections are color-coded throughout — match a bar's color to the stripe above to see where you are.

1 How This Guide Is Built

Andrew Ng teaches this topic bottom-up: motivation first, then a small concrete example, then notation introduced only as it's needed, then two 'what-if' assumptions that get combined into one unified model. This guide follows the same order, but slows down at the two places students usually get stuck — **notation** and the **cost function** — and adds fully worked numeric examples at each step.

1. **Practical motivation** — recommender systems drive a huge share of Amazon/Netflix/food-app revenue, more than their academic attention would suggest.
2. **One running example** — 4 users (Alice, Bob, Carol, Dave) rating 5 movies from 0–5 stars. Every equation in this guide is demonstrated on this same grid.
3. **Incremental notation** — symbols like n_u, n_m, r(i,j) and y(i,j) are introduced only once they're needed (full reference table in Section 2).
4. **Two what-if assumptions, then unified** — first assume movie features x are known and only learn user parameters w, b; then flip it and assume w, b are known to deduce x; finally learn w, b, and x all together with one joint cost function.
5. **Generalize to binary labels** — swap linear prediction for a sigmoid and mean-squared error for cross-entropy loss, exactly the way linear regression generalizes to logistic regression.

2 Notation Reference — Read This First

Every symbol below is used consistently for the rest of the guide. Keep this page open — most confusion in this topic comes from losing track of what a superscript *(i)* vs *(j)* refers to, not from the math itself.

Symbol	Meaning	In the running example
n_u	number of users	$n_u = 4$ (Alice, Bob, Carol, Dave)
n_m	number of movies / items	$n_m = 5$
n	number of features per movie	n = 2 (x_1=romance, x_2=action)
j	index over **users**, j = 1 … n_u	j = 1 means Alice
i	index over **movies**, i = 1 … n_m	i = 1 means “Love at Last”
r(i, j)	1 if user j rated movie i, else 0	r(1,1) = 1, r(3,1) = 0 (Alice hasn't seen it)
y(i, j)	the actual rating user j gave movie i (only defined when r(i,j)=1)	y(1,1) = 5
$x^{(i)}$	feature vector of movie i	$x^{(1)}$ = [0.9, 0.0]
$w^{(j)}$, $b^{(j)}$	parameters (weights, bias) belonging to user j	$w^{(1)}$ = [5, 0], $b^{(1)}$ = 0
$\hat{y}^{(i,j)}$	predicted rating for movie i by user j	$\hat{y}^{(3,1)}$ = 4.95

Symbol	Meaning	In the running example
$m^{(j)}$	number of movies user j has rated	$m^{(1)} = 3$ for Alice
λ	regularization strength (a scalar you choose)	λ = 1 (typical default)

Rating grid y(i,j) — rows = movies i, columns = users j

	Alice (j=1)	Bob (j=2)	Carol (j=3)	Dave (j=4)
Love at Last	5	5	0	0
Romance Forever	5	?	?	0
Cute Puppies of Love	?	4	0	?
Nonstop Car Chases	0	0	5	4
Sword vs Karate	0	0	5	?

The rating grid: rows are movies (i), columns are users (j). A cell holds y(i,j) when r(i,j)=1, and a '?' (r(i,j)=0) when the user hasn't rated it.

Worked example — reading the notation off the grid

- Look at row “Romance Forever”, column Bob: the cell shows **?**, so r(2,2) = 0 and y(2,2) is **undefined** — Bob hasn't rated it yet, so it never enters any sum.
- Look at row “Love at Last”, column Alice: the cell shows 5, so r(1,1) = 1 and y(1,1) = 5.
- $m^{(1)}$ (movies Alice rated) counts the non-'?' cells in her column: Love at Last, Romance Forever, Nonstop Car Chases, Sword vs Karate = 4 rated movies, so $m^{(1)} = 4$.

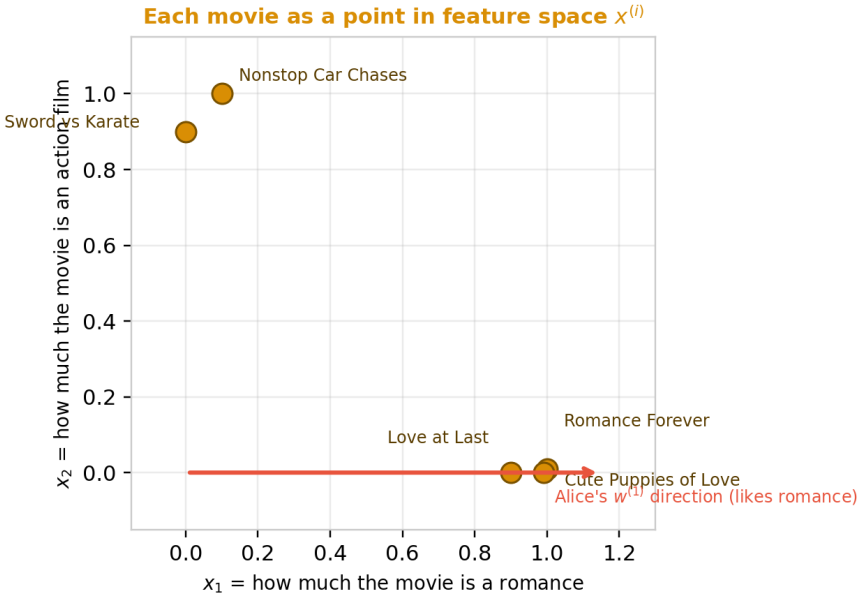
Common notation confusions

- **(j) is not an exponent.** $w^{(j)}$ means “the w that belongs to user j”, not w raised to a power. The same goes for every parenthesized superscript in this topic.
- **Order matters in r(i,j) and y(i,j):** the convention here is movie-first, user-second — r(i,j) asks “did user j rate movie i?”, matching row i / column j in the grid above.
- **ŷ vs y:** $y^{(i,j)}$ is the real rating that already exists in the data; $\hat{y}^{(i,j)}$ is the model's prediction, computed even for cells that are still question marks.

Takeaway: before touching any formula, be able to say out loud what n_u, n_m, r(i,j), y(i,j), $w^{(j)}$, and $x^{(i)}$ each refer to in your own dataset. Every cost function below is just these symbols combined with a sum.

3 Step 1 — Assume We Know the Movie Features

To build intuition, Andrew Ng first assumes we already have descriptive features for every movie: x_1 = how much it's a romance, x_2 = how much it's an action film. Under this assumption, learning each user's taste is **ordinary linear regression, run separately for every user**.



Movies plotted by their feature vector. Notice romance movies cluster near $x_1 \approx 1$, action movies near $x_2 \approx 1$.

Prediction model

$$\hat{y}^{(i,j)} = w^{(j)} \cdot x^{(i)} + b^{(j)}$$

$w^{(j)}$, $b^{(j)}$ belong to user j; $x^{(i)}$ belongs to movie i — this is exactly a linear regression prediction, per user.

Worked example — predicting Alice's rating

- Movie: *Cute Puppies of Love* (i = 3), features $x^{(3)} = [0.99, 0.0]$
- Alice's learned parameters: $w^{(1)} = [5, 0]$, $b^{(1)} = 0$
- $\hat{y}^{(3,1)} = (5 \times 0.99) + (0 \times 0.0) + 0 = \mathbf{4.95}$ — matches her pattern of loving romance, ignoring action.

Cost function for one user

Let $m^{(j)}$ be the number of movies user j has rated. The mean-squared-error cost, with L2 regularization on that user's weights, is:

$$J(w^{(j)},b^{(j)}) = 1/2m^{(j)} \sum_{i:\, r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 \;+\; \lambda/2m^{(j)} \sum_{k=1}^{n} (w_k^{(j)})^2$$

Simplification: because $m^{(j)}$ is just a constant for a fixed user, dividing the whole cost by it never changes *where* the minimum sits — only how steep the bowl looks. So in recommender systems it's dropped for convenience:

$$J(w^{(j)},b^{(j)}) = \tfrac{1}{2} \sum_{i:\, r(i,j)=1} (w^{(j)}\cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 \;+\; \tfrac{\lambda}{2} \sum_{k=1}^{n} (w_k^{(j)})^2$$

the sum $\sum_{i:\, r(i,j)=1}$ only runs over movies this specific user actually rated — never a dense grid sum.

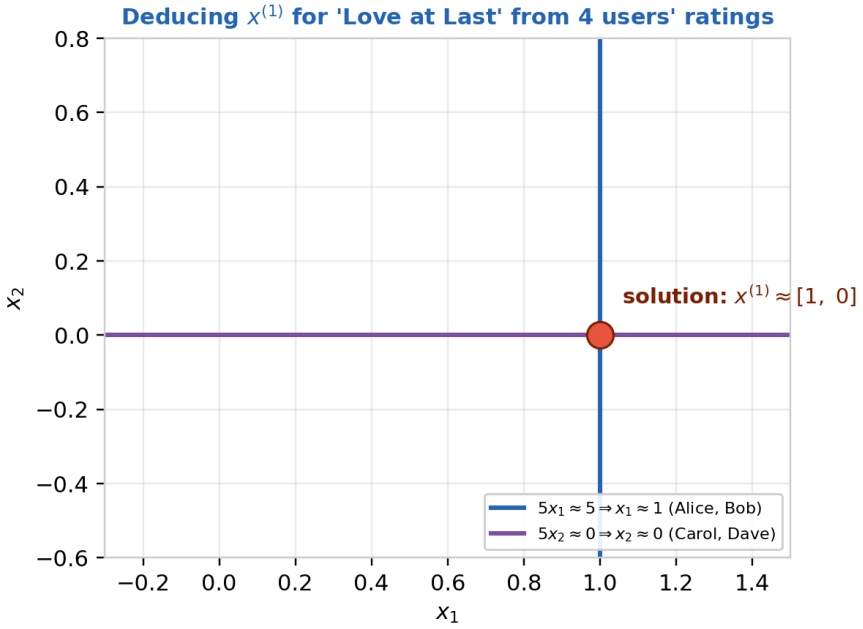
To learn parameters for *every* user at once, just add another sum over j on the outside:

$$J(w^{(1)},b^{(1)},\ldots,w^{(n_u)},b^{(n_u)}) = \tfrac{1}{2} \sum_{j=1}^{n_u} \sum_{i:\, r(i,j)=1} (w^{(j)}\cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \ldots \text{ regularization}$$

movie 1, 2, 3, 4 finished?

4 Step 2 — Learning Movie Features from Scratch

In reality nobody hand-labels "how much romance" is in a movie. So Ng flips the assumption: suppose we **already know** every user's $w^{(j)}$, $b^{(j)}$, and want to **deduce** a movie's features x from the ratings it received.



Two users' constraints intersect at a single point in feature space — that intersection is the deduced $x^{(1)}$.

Worked example — deducing $x^{(1)}$ for "Love at Last"

Known parameters (b ignored, all = 0): Alice/Bob w = [5, 0], Carol/Dave w = [0, 5]. Observed ratings for this movie: Alice=5, Bob=5, Carol=0, Dave=0. We want $x = [x_1, x_2]$ such that every user's prediction roughly matches what they rated:

- Alice: $w^{(1)}\cdot x \approx 5 \;\Rightarrow\; 5x_1 \approx 5$
- Bob: $w^{(2)}\cdot x \approx 5 \;\Rightarrow\; 5x_1 \approx 5$
- Carol: $w^{(3)}\cdot x \approx 0 \;\Rightarrow\; 5x_2 \approx 0$
- Dave: $w^{(4)}\cdot x \approx 0 \;\Rightarrow\; 5x_2 \approx 0$

All four are satisfied by $x^{(1)} \approx [1, 0]$ — which reads back as "almost pure romance, no action", matching the movie's actual genre. This is only possible because **multiple users** rated the same movie; with a single user's data there aren't enough constraints to pin down two unknowns.

Cost function for learning features

To learn the features of one movie i, given everyone's w, b:

$$J(x^{(i)}) = \tfrac{1}{2} \sum_{j:\, r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 \;+\; \tfrac{\lambda}{2} \sum_{k=1}^{n} (x_k^{(i)})^2$$

notice this is the mirror image of the Section 3 cost function — now we sum over users j who rated this one movie, instead of movies one user rated.

And to learn every movie's features at once, sum over all movies too:

$$J(x^{(1)},\ldots,x^{(n_m)}) = \tfrac{1}{2} \sum_{i=1}^{n_m} \sum_{j:\, r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \ldots \text{ regularization}$$

Takeaway: collaborative filtering only works because ratings are shared — the same movie rated by several users gives enough equations to solve for its hidden features, the same way the same user rating several movies gives enough equations to solve for their taste.

5 Step 3 — The Unified Cost Function J(w,b,x)

Sections 3 and 4 each assumed *half* the unknowns were already known. In practice **neither** w,b nor x are known — so instead of alternating between guessing one and solving for the other, we learn all of them **simultaneously** by combining both cost functions into one.

Why the two summation orders are the same sum

Summing "over every user, then every movie that user rated" ($\sum_j \sum_{i:\, r(i,j)=1}$) and summing "over every movie, then every user who rated it" ($\sum_i \sum_{j:\, r(i,j)=1}$) visit the exact same set of cells — every (i,j) pair with a real rating. So both can be written as one flat sum over that pair set:

$$\sum_{j=1}^{n_u} \sum_{i:\, r(i,j)=1} (\ldots) \;=\; \sum_{i=1}^{n_m} \sum_{j:\, r(i,j)=1} (\ldots) \;=\; \sum_{(i,j):\, r(i,j)=1} (\ldots)$$

The joint cost function

$$J(w,b,x) = \tfrac{1}{2} \sum_{(i,j):\, r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 \;+\; \tfrac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^{n} (w_k^{(j)})^2 \;+\; \tfrac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^{n} (x_k^{(i)})^2$$

Three pieces, and each one has one job:

Term	What it does
$\sum_{(i,j)} (\text{pred} - \text{actual})^2$	penalizes prediction error, summed only over cells that actually have a rating

Term	What it does
$\lambda\sum(w_k^{(j)})^2$	regularization — keeps every user's weights from growing huge (overfitting to few ratings)
$\lambda\sum(x_k^{(i)})^2$	regularization — keeps every movie's learned features from growing huge

Gradient descent updates all three unknowns together

$$w^{(j)} \leftarrow w^{(j)} - \alpha\, \partial J/\partial w^{(j)} \qquad b^{(j)} \leftarrow b^{(j)} - \alpha\, \partial J/\partial b^{(j)} \qquad x^{(i)} \leftarrow x^{(i)} - \alpha\, \partial J/\partial x^{(i)}$$

*α is the learning rate. The key shift from ordinary linear regression: $x^{(i)}$ is now also a **parameter** being updated, not a fixed input feature.*

5b Worked Numeric Example — Computing J(w,b,x) by Hand

A tiny 2-user, 2-movie grid with 1 missing rating, using a single feature (n=1) so every number can be checked by hand. This isolates exactly how the three terms above turn into a single number.

Given	Value
User 1: $w^{(1)}$, $b^{(1)}$	2, 0
User 2: $w^{(2)}$, $b^{(2)}$	1, 1
Movie 1: $x^{(1)}$	1.5
Movie 2: $x^{(2)}$	0.5
Ratings y(i,j)	y(1,1)=4, y(1,2)=2, y(2,2)=1, y(2,1)=? (missing)
λ	1

Step A — predictions and squared errors (only where r(i,j)=1)

(i,j)	ŷ = w·x+b	y	error	error²
(1,1)	2×1.5 + 0 = 3.0	4	−1.0	1.00
(1,2)	1×1.5 + 1 = 2.5	2	0.5	0.25
(2,2)	1×0.5 + 1 = 1.5	1	0.5	0.25
(2,1)	— skipped, r(2,1)=0 —	?	—	—

Sum of squared errors = 1.00 + 0.25 + 0.25 = 1.50. First term = · 1.50 = 0.75 (that's the outside 1/2).

Step B — regularization on w

$$\lambda/2 \sum(w_k^{(j)})^2 = 1/2\ (2^2 + 1^2) = 1/2\ (4+1) = \mathbf{2.5}$$

Step C — regularization on x

$$\lambda/2 \sum(x_k^{(i)})^2 = 1/2\ (1.5^2 + 0.5^2) = 1/2\ (2.25+0.25) = \mathbf{1.25}$$

Total

$$J(w,b,x) = 0.75 + 2.5 + 1.25 = \mathbf{4.5}$$

That single number, 4.5, is what gradient descent nudges downward on every iteration — by adjusting w, b, and x a little in the direction that shrinks all three terms.

Same computation, as NumPy — sanity-checks to 4.5

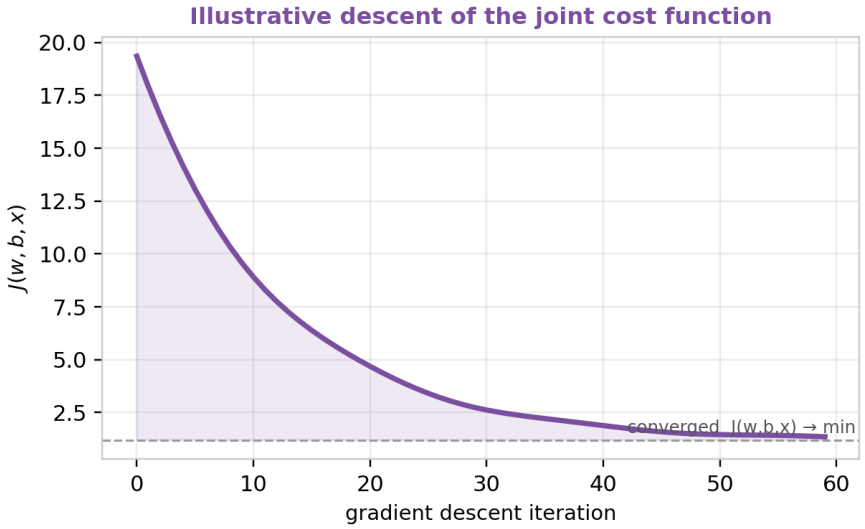
PYTHON

```python
import numpy as np

# X: movie features, shape (n_movies, n_features)
X = np.array([[1.5], [0.5]])
# W, b: user parameters
W = np.array([[2.0], [1.0]])
b = np.array([0.0, 1.0])
# Y: ratings grid, R: 1 if rated else 0 (0 is a placeholder in Y)
Y = np.array([[4.0, 2.0], [0.0, 1.0]])
R = np.array([[1, 1], [0, 1]])
lambda_ = 1.0

def cost_function(X, W, b, Y, R, lambda_):
    predictions = X @ W.T + b          # shape (n_movies, n_users)
    err = (predictions - Y) * R        # zero out unrated cells
    J = 0.5 * np.sum(err ** 2)
    J += (lambda_ / 2) * np.sum(W ** 2)
    J += (lambda_ / 2) * np.sum(X ** 2)
    return J

print(cost_function(X, W, b, Y, R, lambda_))   # -> 4.5
```



Illustrative shape of J(w,b,x) decreasing over gradient descent iterations as w, b, and x are all updated together.

Takeaway: the joint cost function is nothing more than (prediction error, summed only over rated cells) + (a penalty so w doesn't grow too large) + (a penalty so x doesn't grow too large). Compute each of the three pieces separately, then add.

6 Step 4 — Generalizing to Binary Labels

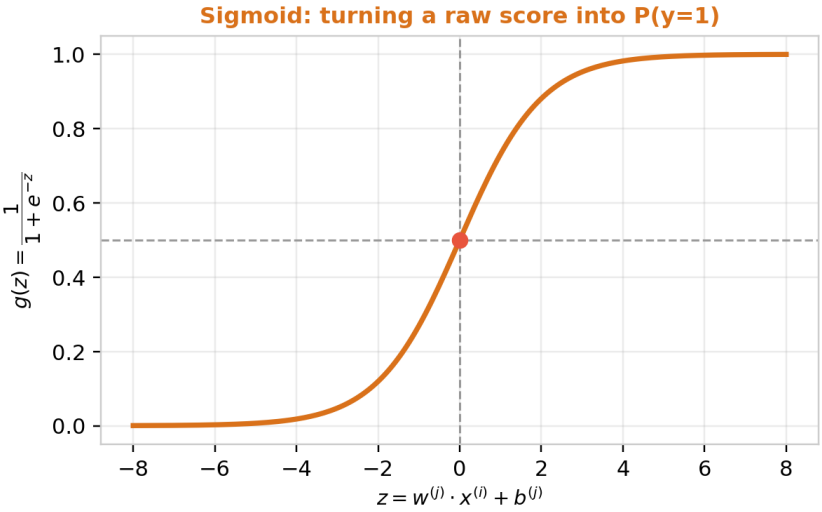
Most real systems don't get 1–5 star ratings — they get binary signals: clicked or not, purchased or not, watched ≥ 30s or not. The same joint model generalizes here exactly the way linear regression generalizes into logistic regression.

Signal	y = 1 means	y = 0 means	? means
Online shopping	purchased item after seeing it	shown, not purchased	not shown yet
Social media	liked / favorited	shown, not liked	not shown yet
Watch time	watched ≥ 30 seconds	shown, watched < 30s	not shown yet
Ad click-through	clicked the ad	shown, not clicked	not shown yet

The critical distinction: **0 is a real, informative label** (shown but ignored), not a missing value — only '?' is missing.

From raw score to probability: the sigmoid

$$P(y^{(i,j)}=1) = g(w^{(j)} \cdot x^{(i)} + b^{(j)}) \quad \text{where} \quad g(z) = 1/(1+e^{-z})$$



The sigmoid squashes any raw score z into a probability between 0 and 1, centered at 0.5 when z=0.

Worked example — turning a score into a probability

- Suppose $w^{(j)} \cdot x^{(i)} + b^{(j)} = 1.2$ for some user/movie pair.
- $g(1.2) = 1 / (1+e^{-1.2}) \approx$ **0.768** — the model predicts a 76.8% chance this user engages with this item.

Cost function: binary cross-entropy

Instead of squared error, each rated pair contributes a cross-entropy loss:

$$L(\hat{y}^{(i,j)}, y^{(i,j)}) = -y^{(i,j)} \log(\hat{y}^{(i,j)}) - (1-y^{(i,j)}) \log(1-\hat{y}^{(i,j)})$$

Worked continuation of the example above (prediction 0.768):

- If the true label is y=1: loss = $-\log(0.768) \approx$ **0.264** (small, since the model was right to lean toward 1).
- If the true label is y=0: loss = $-\log(1-0.768) = -\log(0.232) \approx$ **1.461** (much bigger — the model was confidently wrong).

Summed over every rated pair, plus the same regularization as before:

$$J(w,b,x) = \sum_{(i,j):\ r(i,j)=1} L(\hat{y}^{(i,j)}, y^{(i,j)}) + \text{regularization}$$

Takeaway: the only two changes from the star-rating model are: (1) the raw score w·x+b gets passed through the sigmoid before comparing to the label, and (2) squared error is replaced with cross-entropy loss. The regularization terms and the sum-over-rated-pairs structure stay identical.

7 Interview Prep & Real-World Applications

Q1. Why can we drop $1/m^{(j)}$ from the single-user cost function without changing the answer?

Because $m^{(j)}$ is a constant for a fixed user — scaling a function by a positive constant moves its value up or down but never moves the location of its minimum. Dropping it just simplifies the implementation.

Q2. Why can't ordinary single-user linear regression learn features from scratch, while collaborative filtering can?

One user's ratings give one equation per rated movie but two unknowns per movie (x_1, x_2) — underdetermined. Multiple users rating the same movie contribute multiple independent equations for that movie's features, which is what makes solving for them possible.

Q3. What's the practical difference between explicit and implicit feedback, and how does the cost function change?

Explicit feedback is a direct rating (stars); implicit feedback is inferred from behavior (clicks, dwell time, purchases). The model swaps the linear prediction for a sigmoid and mean-squared error for binary cross-entropy — everything else (the sum over rated pairs, the regularization terms) is unchanged.

Where this shows up in industry

Application	How collaborative filtering is used
Netflix / Disney+	predict star ratings for unwatched titles, recommend the highest-predicted ones
Amazon	discover hidden product similarities from co-purchase and rating behavior, no manual tagging needed
Google / Meta Ads	binary click-through prediction — clicked (1), shown-not-clicked (0), not shown (?)

8 Foundational vs. Supplementary, and What to Do Next

Foundational — must internalize

- The meaning of n_u, n_m, r(i,j), y(i,j) (Section 2).
- The joint cost function J(w,b,x) and what each of its three terms does (Section 5).
- How gradient descent updates w, b, *and* x simultaneously (Section 5).
- The binary-label extension via sigmoid + cross-entropy (Section 6).

Supplementary — pedagogical stepping stones

- Assuming features are known first (Section 3), or assuming parameters are known first (Section 4) — these are teaching devices to build intuition, not how the algorithm is actually initialized in practice.

Commonly tricky areas

- The sum $\sum_{(i,j):\, r(i,j)=1}$ is **not** a dense grid sum — it only ever touches cells with an actual rating.
- Both $w^{(j)}$, $b^{(j)}$ *and* $x^{(i)}$ are learned parameters here — unlike ordinary linear regression, the "features" change during training too.
- In binary systems, 0 is a real observed label (shown, ignored) — only '?' means missing/not-yet-shown.

Concrete next steps

1. By hand: extend the Section 5 worked example to a full gradient descent step — compute $\partial J/\partial w^{(1)}$ and take one update with $\alpha=0.01$.
2. Implement: write a NumPy version of J(w,b,x) and its gradients for the 4-user, 5-movie running example.
3. Practice: work through how implicit feedback (30-second watch time) changes preprocessing before training, compared to explicit star ratings.

Takeaway: if you can (1) name every symbol in J(w,b,x) without looking it up, and (2) reproduce the Section 5 numeric example on a blank page, you've internalized the two hardest parts of this topic.